

1. Overview

OLED (Organic Light-Emitting Diode) displays are popular in embedded systems because they produce high-contrast images without a backlight, consume very little power, and are easy to interface over I2C or SPI. The **SSD1306** controller is the most widely used driver IC for small monochrome OLEDs in the hobbyist and prototyping space.

This guide covers a complete test suite that exercises every major capability of the SSD1306: pixel addressing, geometric primitives, text rendering, hardware commands (invert, contrast, scroll), and stress patterns (checkerboard, noise). Four platform implementations are provided — Arduino, Raspberry Pi (Python), MicroPython (ESP32/ESP8266), and STM32 HAL (C) — so you can start testing regardless of your hardware.

Why Test an OLED?

- Verify hardware soldering and connections before integration into a larger project.
- Identify dead or stuck pixels that would affect the user interface.
- Confirm the I2C address and communication speed are configured correctly.
- Validate that fonts, graphics primitives, and animations behave as expected.
- Stress-test the display controller and power supply stability.

2. Hardware & Wiring

SSD1306 Pin Description

Pin	Name	Description
1	GND	Ground (0 V reference)
2	VCC	Power supply: 3.3V or 5V depending on module
3	SCL	I2C clock line (100 kHz standard / 400 kHz fast mode)
4	SDA	I2C data line (bidirectional)

Platform Wiring Quick-Reference

OLED Pin	Arduino Uno	Raspberry Pi	ESP32	ESP8266	STM32F4
VCC	3.3V / 5V	Pin 1 (3.3V)	3.3V	3.3V	3.3V
GND	GND	Pin 6 (GND)	GND	GND	GND
SDA	A4	Pin 3 (GPIO2)	GPIO21	GPIO4	PB7 (I2C1)
SCL	A5	Pin 5 (GPIO3)	GPIO22	GPIO5	PB6 (I2C1)

Note: Always use 3.3V for ESP8266 and ESP32. Most Arduino-compatible OLED modules include level shifting and work fine at 5V.

3. Test Pattern Reference

The suite includes 13 test patterns. Each targets a specific aspect of display function:

#	Test Name	Purpose	Pass Criteria
0	White Fill	All pixels ON simultaneously	Uniform bright screen, no dark patches
1	Black Fill	All pixels OFF	Completely dark screen; no stuck-on pixels
2	Checkerboard	Alternating pixel ON/OFF	Clean 1x1 chess pattern, no bleed
3	Text Rendering	Font glyph accuracy at all sizes	Readable characters, correct spacing
4	Lines	Bresenham line algorithm test	Straight, unbroken diagonals and axes
5	Rectangles	Outline and fill rects	Sharp corners, filled area solid
6	Circles	Arc / trigonometric pixel placement	Smooth circle, no gaps in outline
7	Progress Bar	Composite UI element (rect + text)	Bar fills evenly, % label correct
8	Sine Wave	Floating-point rendering fidelity	Smooth continuous wave across width
9	Pixel Noise	Random pixel addressing (400 pixels)	Scattered dots, no column/row failure
10	Grid	8-pixel uniform grid lines	Even spacing, intersections clean
11	Invert	Hardware display invert (SSD1306 cmd)	Colors flip without data loss
12	Contrast Sweep	Contrast register 0–255 ramp	Smooth brightness ramp, no flicker

4. Platform Setup & Usage

4.1 Arduino (SSD1306 128x64 via I2C)

File: `oled_test_arduino.ino`

Required Libraries (install via Library Manager):

- Adafruit SSD1306
- Adafruit GFX Library

Setup steps:

- Wire the OLED per the table in Section 2.
- Open the .ino file in the Arduino IDE.
- Select your board and COM port.
- Click Upload. Open Serial Monitor at 115200 baud to see test progress.

Note: If the display does not initialise, try changing `SCREEN_ADDRESS` from `0x3C` to `0x3D`.

4.2 Raspberry Pi (Python 3 + luma.oled)

File: `oled_test_pi.py`

Install dependencies:

```
sudo apt install python3-pip python3-pil -y
pip3 install luma.oled Pillow
```

Enable I2C:

```
sudo raspi-config
# Interface Options -> I2C -> Enable
sudo reboot
# Verify: i2cdetect -y 1 (should show 0x3c)
```

Run:

```
python3 oled_test_pi.py
```

Note: If `luma.oled` is not installed the script runs in SIMULATION mode and saves PNG images instead of driving the display.

4.3 MicroPython (ESP32 / ESP8266)

File: **oled_test_micropython.py**

The **ssd1306** module is bundled in official MicroPython firmware. No extra packages needed.

Flash to device:

```
# Using mpremote:
mpremote cp oled_test_micropython.py :main.py
mpremote reset

# Using ampy:
ampy --port /dev/ttyUSB0 put oled_test_micropython.py main.py
```

Edit **I2C_SDA** and **I2C_SCL** constants at the top of the file to match your board's pin layout before flashing.

4.4 STM32 HAL (C)

File: **oled_test_stm32.c**

Depends on Olivier Van den Eede's portable SSD1306 HAL driver:

```
https://github.com/afiskon/stm32-ssd1306
```

Integration steps:

- Generate a project with STM32CubeMX. Enable I2C1 in Fast Mode (400 kHz).
- Copy `ssd1306.c` / `ssd1306.h` into your `Src` / `Inc` directories.
- Add `oled_test_stm32.c` to your project.
- Call `OLED_RunTestSuite()` from `main()` after all peripheral initialisations.
- Build and flash with ST-Link / OpenOCD.

5. Troubleshooting

Symptom	Likely Cause	Fix
Display stays blank / dark	Wrong I2C address or no power	Run i2cdetect; try 0x3D; check VCC/GND
Garbled pixels on startup	Missing pull-up resistors	Add 4.7 kΩ pull-ups on SDA and SCL to VCC
Only top half renders	Wrong display height in config	Set SCREEN_HEIGHT to 64 (not 32)
Flicker during animation	display() called too frequently	Reduce refresh rate; batch draw calls
Text shows boxes / squares	Unicode chars in font not supported	Use ASCII only; avoid subscript Unicode
I2C timeout errors	Speed too high or long wires	Drop to 100 kHz; use shorter cables
ESP8266 reboot loop	I2C pins conflict with boot strapping	Use GPIO 4 (SDA) and GPIO 5 (SCL) only
Dim display even at max brightness	Capacitor pump not fully charged	Add 100uF cap on VCC; check PSU current

6. SSD1306 Key Commands

These raw I2C commands can be sent directly when you need fine control beyond what a library exposes:

Hex Byte	Command	Notes
0xAE	Display OFF	Sleep mode — panel off, RAM retained
0xAF	Display ON	Wake up from sleep
0xA6	Normal Display	1 = lit pixel
0xA7	Inverted Display	0 = lit pixel (hardware invert)
0x81, val	Set Contrast	val 0x00–0xFF; default ~0x7F
0xA4	Entire Display OFF	Override RAM; all pixels off (test)
0xA5	Entire Display ON	Override RAM; all pixels on (test)
0x26/0x27	Horizontal Scroll	Right / Left continuous scroll
0x2E	Stop Scroll	Freeze scrolling
0x2F	Start Scroll	Begin scroll after 0x26/0x27 setup
0x20, 0x00	Horizontal Addr Mode	Wrap columns then pages
0x20, 0x01	Vertical Addr Mode	Wrap pages then columns
0x20, 0x02	Page Addr Mode	Default; manual page select needed

7. File Summary

Filename	Platform	Language	Key Library
oled_test_arduino.ino	Arduino	C++	Adafruit_SSD1306, Adafruit_GFX
oled_test_pi.py	Raspberry Pi	Python 3	luma.oled, Pillow

oled_test_micropython.py	ESP32 / ESP8266	MicroPython	ssd1306 (built-in)
oled_test_stm32.c	STM32	C (HAL)	stm32-ssd1306 (afiskon)
OLED_Testing_Guide.pdf	All	—	This document